Marshmallow formalization by prompt design

Xinze Li* Simone Severini†

August 6, 2026

Abstract

As AI-assisted Lean formalizations emerge where authors write little or no code, the human role shifts toward prompt design and mathematical judgment. In this paradigm, treating code as the primary mathematical artifact requires some methodological discipline: the standards we hold code to as software (readability, naming, structural discipline) must carry over to it as mathematics. We present an approach with two aims: non-mathematical engineering overhead is separated out so it can be managed on its own, and the prose serves only as an interface to the formal development, which carries the mathematics itself. We validate this approach through a mathematical "marshmallow"—a small but intellectually rewarding problem—by providing a publicly available Lean 4 case study in combinatorial matrix theory.

1 Introduction

AI tools can now produce substantial Lean developments under human prompt design [Ili26, Avi26]. This shifts the practical question of how a formalization is built, and with it, the methodological question of what a formalization is actually for. Lean is built on dependent type theory, in which the Curry-Howard correspondence identifies proofs with terms and mathematical statements with types [MU21]. In this sense, the formal development of a proof is the proof itself. The major formalization successes of the last five years [Liq22, vDMN23, ABB+23, BdFFD+25, BBB+25, BT25] have been carried out alongside reflective accounts of the practice of building such developments [Avi23, Avi26].

We report a case study that takes the Curry-Howard correspondence as its working premise: the formal Lean development is the mathematics, and the prose only renders it. There is no technical novelty in what we do, as there is now a growing literature that examines the methodological consequences of treating formal developments as designed mathematical artifacts [CT24, BBC+26, vDEL20, ACMT25, Ili26]. But Section 2 looks at the practices that naturally follow from this constraint. A network analysis of `Mathlib` [LPSS26] shows its structure is heavily infrastructural (*e.g.*, coercions and typeclasses) rather than strictly mathematical. The practices below apply this library-wide distinction to individual developments. We tag every declaration as mathematical content or *engineering tax*, that is, scaffolding with no mathematical counterpart. We gather this overhead into a standalone Foundation layer that the mathematical layer reaches only through clean interfaces. We enforce a naming discipline under which every declaration reads aloud as a mathematical statement. Under this arrangement the authors write no code, but they steer the formalization entirely through prompt design and mathematical judgment. The central principle is as follows:

the code must read as mathematics.

*Department of Mathematics, University of Toronto
†Department of Computer Science, UCL

As a case study, we characterize the digraphs of real-orthogonal upper Hessenberg matrices. A *marshmallow*, in Erdős's sense, is a small result that offers a moment of intellectual reward but no deep mathematical consequences. (The choice of problem here is driven by the second author's curiosity, returning after twenty-five years to a problem from their doctoral years [Sev03], and by an honest assessment of their mathematical strength, then and now.) Nothing in the methodology discussed here depends on the problem being a marshmallow. If anything, keeping engineering tax out of the mathematical narrative matters more in a large formalization, since that is where the narrative is easiest to lose.

Below, Section 2 sets out the methodology, and Section 3 presents the case study, with direct cross-references into the Lean development. The full development is available on GitHub. Every theorem and definition below carries a link into its source.

We have also published the mathematics of Section 3 on `astrolabe.network`, where every statement of that section is cut into a card addressed by a unique hash, and the relations between cards are themselves nodes, each carrying the semantics of the relation it stands for. The visualization and the network analyses it carries follow [LPSS26]. The design was initially set out in [Li26], with many updates since; the submission format and process will follow.

2 Formalization as the primary artifact

In dependent type theory, a Lean formalization is inherently a proof [MU21], requiring no accompanying prose. On this basis, we treat the Lean development as the primary mathematical artifact of this project, and the prose as merely one rendering of it. Because the human contribution shifts from writing code to exercising mathematical judgment and steering the AI, the central object of study of this paper is precisely prompt design. Getting the proof to compile is the starting point of that work, not its endpoint. What distinguishes a primary mathematical artifact from a mere verification record lies entirely past that threshold: it depends on whether the code, in its own form, legibly conveys its mathematical content. Finally, we acknowledge that neither author has a formal software engineering background. We treat AI tools as informal mentors in software practice, working to elevate this project to structural standards we did not initially command, and we expect our execution will still reflect this learning curve.

Prompt example — project bootstrap.

> *The goal is not just proving the main theorem; it is producing code whose main theorem signature reads as a literal translation of the paper sentence. Engineering tax should be separated from mathematical content and encapsulated in independent modules; every name should carry clear mathematical semantics. Survey the paper's core mathematical objects; propose a Foundation-structure design. Output the design only; do not implement until I confirm.*

We refer to type-theoretic overhead with no paper analogue as *engineering tax*. Left unmanaged, this tax sits mixed among mathematical-layer declarations and muddles the mathematical narrative. Three forms recur in this project:

- bound-carrying boilerplate, where anonymous-constructor expressions bundle a value together with proofs that it lies in range;
- index translations, where the formal system's natural index types differ from the paper's mathematical indexing conventions;
- case analyses on small numeric ranges, where boundary cases dismissed by the paper in a single sentence require explicit, separate handling.

A typical instance is the lemma `cyclic_shift_arc_forward`: its statement reads like a standard mathematical theorem, while its proof body spans roughly 80 lines combining all three forms of overhead. A casual reading of the code easily overlooks the true density of this burden.

To surface this hidden overhead, we tag every declaration as `Math`, `Eng`, or `Mixed`. Because these tags are formatted as docstrings above each declaration, they travel inherently with the code:

```
/-! ## Mathematical layer -- paper-side vertex type. A `Vertex n`
    is a label `v ∈ {1, ..., n}` (Sec. 0 of the paper). -/

/-! ## Engineering layer -- Type B bridge: `Vertex n ↔ Fin n`
    (paper 1-indexed ↔ Lean 0-indexed). -/
```

We enforce this tagging discipline with a small project-specific linter, alongside Mathlib's standard tooling [mC20] (naming, docstrings, style, plus `shake` for unused imports), ensuring that violations surface mechanically.

Engineering tax cannot be eliminated, but its location can be chosen. We encapsulate it in independent modules so the mathematical layer interacts with them strictly through clean APIs. This applies Parnas's information-hiding principle [Par72] to formalization: each module hides one engineering concern from the mathematical layer. The core of this effort is a small Foundation layer of bundled structures, each encapsulating a paper-side mathematical object as a Lean type [Baa24, CT24, Avi17]. The layer introduces `Vertex`, `ActiveSet`, `Digraph`, `OrthogonalHessenberg`, `UnreducedAngles`, and others—allowing each structure to absorb the corresponding combination of hypotheses just once. Three further modules (`Singleton.Fin`, `EntryAccess`, `PartialProduct`) are factored out by thematic content.

Such structural refactors involve large-scale cascades—the largest spanned 12 files and roughly 232 call sites—where the design judgment is made by humans and the execution is carried out by AI. To illustrate the impact, consider the Bridge theorem signature before the structural pass:

```
theorem digraph_model (hn : 2 ≤ n) (θ : Fin (n - 1) → ℝ)
    (hunred : ∀ k, Real.sin (θ k) ≠ 0) :
    ∀ i j : Fin n,
      givensProduct θ i j ≠ 0 ↔ Arc n (activeSet θ) (i.val + 1) (j.val + 1)
```

After the structural pass:

```
theorem digraph_model (hn : 2 ≤ n) (θ : UnreducedAngles n) :
    θ.product.supportDigraph = θ.activeSet.digraph
```

The engineering tax has not disappeared; it simply now lives within the Foundation modules. Consequently, the formal Bridge theorem reads exactly as the natural-language paper sentence.

Prompt example — abstraction design.

> *Scan all uses of* ***`Fin n`*** *that represent paper-side vertices. Report the count of anonymous-mk constructions, the count of* 0*-indexed-to-*1*-indexed translations, and the most common usage patterns. From the data, propose a* ***`Vertex n`*** *structure with constructors, comparison, neighbouring, and a round-trip API to* ***`Fin n`****. Output the design only.*

Once the engineering tax is isolated, the mathematical layer can carry semantics directly: names become clear mathematical statements, symbols match the paper, and the main theorem signature reads as a literal translation of its natural-language counterpart.

For naming, we apply a simple heuristic: read the name aloud and check whether it forms a complete mathematical thought. A name that fails this test usually signals that the declaration's role has not been fully resolved—it may be too specific, too abstract, or in the wrong place. For instance, `extended_diag_isPlusMinusOne` fails because it leaves the subject ambiguous; revised to `boundaryAppended_diag_isPlusMinusOne`, it forms a clear sentence. This test serves both human and AI readability. Placeholder names like `aux_lemma_3` leave the AI with no semantic basis for judgment in search and completion, whereas a name that encodes its own statement exposes the underlying mathematics directly to the model.

We use the form of the main theorem itself as the project's completion criterion: when the apex signature reads exactly as the paper sentence, the work is done. A green build is necessary, but not sufficient. Consider the apex theorem immediately after proof completion, before the engineering tax has been extracted:

```
theorem classification {n : ℕ} (hn : 2 ≤ n) :
    (∀ S : Finset ℕ, ValidS n S → ∃ θ : Fin (n-1) → ℝ,
        (∀ k, Real.sin (θ k) ≠ 0) ∧ activeSet θ = S) ∧
    (∀ θ : Fin (n-1) → ℝ, (∀ k, Real.sin (θ k) ≠ 0) →
        ∀ i j : Fin n, givensProduct θ i j ≠ 0 ↔
          Arc n (activeSet θ) (i.val + 1) (j.val + 1)) ∧
    (∀ S S' : Finset ℕ, ValidS n S → ValidS n S' →
        2 ≤ S.card → 2 ≤ S'.card → DigraphIso n S S' → S = S')
```

Here, engineering overhead sits alongside mathematical content in the signature, forcing the reader to mentally filter it out. The code is mathematically verified, but it has not assumed responsibility for legibility. Contrast this with the signature after the structural pass:

```
theorem classification {n : ℕ} (hn : 2 ≤ n) :
    (∀ S : ActiveSet n, ∃ θ : UnreducedAngles n, θ.activeSet = S) ∧
    (∀ θ : UnreducedAngles n, θ.product.supportDigraph = θ.activeSet.digraph) ∧
    (∀ S S' : ActiveSet n, 2 ≤ S.elements.card → 2 ≤ S'.elements.card →
        S.digraph ≅ S'.digraph → S = S') :=
  ⟨completeness, digraph_model hn, fun S S' ⇒ rigid S S' hn⟩
```

Every conceptual object in the signature is now packaged inside its own structure. The entire declaration reads as a direct translation of Theorem 3.13: for $n \geq 2$, every active set arises from some unreduced angle vector; the support digraph of the Givens product equals the one given by the active set; and for $|S|, |S'| \geq 2$, digraph isomorphism implies $S = S'$.

Both signatures encode identical mathematical content; the difference lies entirely in whether the code assumes responsibility for its own legibility. Only when the formal development is treated as the primary mathematical artifact can this standard be posed and met.

Prompt example—naming review.

> *Apply the natural-language reading test to every helper in this file. For each name that fails, identify the failure mode (missing subject, placeholder word, unexpanded abbreviation) and propose a revised name that states what the object is, what it is built from, and what property it has. Report the rename list; confirm before implementing.*

3 Case study: Digraphs of real-orthogonal upper Hessenberg matrices

We work with $n \times n$ real matrices ($n \geq 2$), indexed by $[n] = \{1, 2, \ldots, n\}$.

Definition 3.1 (Orthogonal upper Hessenberg matrix)**.** [✓ `OrthogonalHessenberg`]

A matrix $Q \in \mathbb{R}^{n \times n}$ is *upper Hessenberg* if $Q_{ij} = 0$ whenever $j + 1 < i$, and *orthogonal* if $Q^\top Q = I$. We write $\mathcal{H}_n$ for the set of orthogonal upper Hessenberg matrices in $\mathbb{R}^{n \times n}$.

Definition 3.2 (Support digraph)**.** [✓ `supportDigraph`]

The *support digraph* of an $n \times n$ real matrix Q, denoted $D(Q)$, is the directed graph on $[n]$ with an arc $i \to j$ whenever $Q_{ij} \neq 0$. Figure 1 illustrates a small instance.

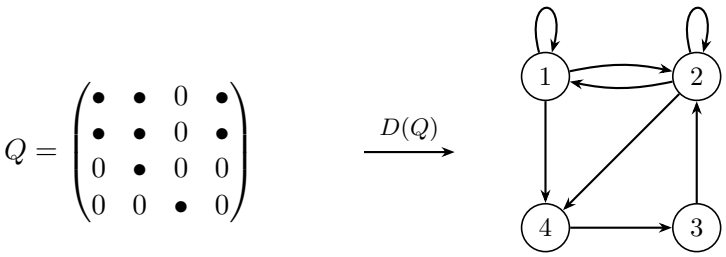

Figure 1: A 4×4 upper Hessenberg matrix ($\bullet$ = nonzero) and its support digraph $D(Q)$. Each nonzero entry Q_{ij} becomes a directed arc $i \to j$.

Definition 3.3 (Sign-equivalence)**.** [✓ `SignEquiv`]

Two matrices $Q, Q' \in \mathbb{R}^{n\times n}$ are *sign-equivalent*, written $Q \simeq_{\text{sign}} Q'$, if $Q' = D_L\, Q\, D_R$ for some *signature matrices* D_L, D_R — diagonal matrices whose diagonal entries are all ± 1.

Corollary 3.4 (Sign-invariance of the support digraph)**.** [✓ `supportDigraph_eq_of_signEquiv`]

Sign-equivalent matrices have identical zero / non-zero patterns; hence $D(Q) = D(Q')$ whenever $Q \simeq_{\text{sign}} Q'$.

Definition 3.5 (Givens rotation, ordered Givens product)**.** [✓ `givensRotation`, ✓ `givensProduct`]

For $k \in \{0, \dots, n-2\}$ and $\theta \in \mathbb{R}$, the *Givens rotation* $G_k(\theta) \in \mathbb{R}^{n\times n}$ is the identity with the 2×2 block at rows / columns $(k, k{+}1)$ replaced by $\left[\begin{smallmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{smallmatrix}\right]$. The *ordered Givens product* of an angle vector $\theta = (\theta_0, \dots, \theta_{n-2})$ is

$$Q_n(\theta) \;:=\; G_0(\theta_0)\, G_1(\theta_1) \cdots G_{n-2}(\theta_{n-2}).$$

The product $Q_n(\theta)$ is always orthogonal upper Hessenberg, and its $(k{+}1, k)$ subdiagonal entry equals $\sin\theta_k$.

The angle vector is called *unreduced* if every $\sin\theta_k$ is non-zero; equivalently, every subdiagonal entry of $Q_n(\theta)$ is non-zero. We write Θ_n for the set of such unreduced angle vectors.

Definition 3.6 (Active set, combinatorial digraph)**.** [✓ `ActiveSet`, ✓ `digraph`]

An *active set* is any subset $S \subseteq [n-1]$. From S we derive

$$R(S) \;:=\; \{1\} \cup \{k+1 : k \in S\}, \qquad C(S) \;:=\; S \cup \{n\}$$

(*active rows / active columns*), and the *combinatorial digraph* $D_n(S)$ on $[n]$, whose arcs are of two kinds: *spine* arcs $j+1 \to j$ for $1 \le j \le n-1$, and *overlay* arcs $i \to j$ whenever $i \in R(S)$, $j \in C(S)$, and $i \le j$. This construction uses the set S alone; no matrix or angle vector enters.

The *active set of an angle vector* $\theta \in \Theta_n$ is

$$S(\theta) \;:=\; \left\{\, k+1 \;:\; k \in \{0, \dots, n-2\},\ \cos\theta_k \neq 0 \,\right\} \;\subseteq\; [n-1],$$

the subset to which D_n is applied on the matrix side.

Example 3.7 ($n = 6$, $S = \{2, 4\}$ — anatomy of $D_6(\{2, 4\})$)**.** [✓ `example_D6_S24`]

The angle vector $\theta = (\pi/2, \pi/4, \pi/2, \pi/4, \pi/2)$ produces $\cos\theta_k \neq 0$ exactly for $k \in \{1, 3\}$, hence $S = \{2, 4\}$. The active rows and columns are

$$R = \{1, 3, 5\}, \qquad C = \{2, 4, 6\},$$

and the resulting digraph $D_6(\{2, 4\})$ is depicted in Figure 2: the spine cycle in solid black and the overlay arcs as dashed blue edges from R to C. Here $R \cap C = \emptyset$, so $D_6(\{2, 4\})$ has no self-loops.

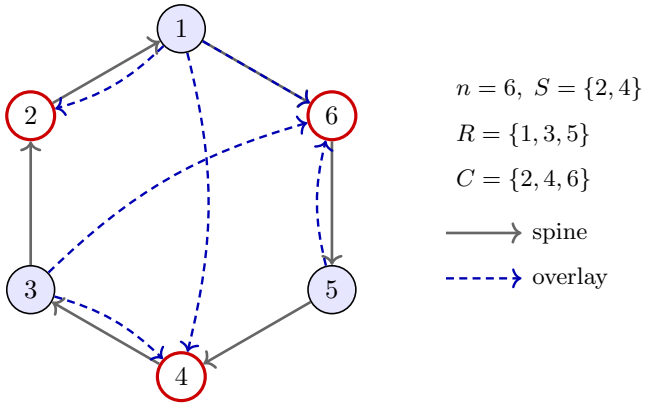

Figure 2: Anatomy of $D_6(\{2, 4\})$. Solid arrows form the directed Hamilton cycle (spine). Dashed blue arrows are the overlay arcs from active rows $R = \{1, 3, 5\}$ (blue fill) to active columns $C = \{2, 4, 6\}$ (red outline). Self-loops occur exactly at vertices in $R \cap C$; here $R \cap C = \emptyset$, so $D_6(\{2, 4\})$ is loopless.

Theorem 3.8 (Universality)**.** [✓ `exists_givensFactorization`]

Every unreduced $Q \in \mathcal{H}_n$ is sign-equivalent to a canonical Givens product: there exists $\theta \in \Theta_n$ with $Q \simeq_{\text{sign}} Q_n(\theta)$.

Proof sketch. Induction on n. Right-multiply Q by the transpose of a Givens factor chosen to clear the last row to $e_n^\top$; orthogonality forces the last column to e_n as well, so the principal $(n-1) \times (n-1)$ corner is itself orthogonal upper Hessenberg, and the induction hypothesis applies. The accumulated sign discrepancies along the inductive peel-off are absorbed into the diagonal D_L, D_R of the sign-equivalence relation. □

Theorem 3.9 (Bridge)**.** [✓ `digraph_model`]

For every $\theta \in \Theta_n$, the matrix support agrees with the combinatorial digraph:

$$D\big(Q_n(\theta)\big) \;=\; D_n\big(S(\theta)\big).$$

Proof sketch. Two inclusions, both stated entry-wise on $Q_n(\theta)_{ij}$. The subdiagonal entries are $\sin\theta_j \neq 0$ by the unreduced hypothesis, exhibiting all spine arcs. For an upper-triangular entry $i \leq j$, a row / column-vanishing analysis on the Givens product shows that $Q_n(\theta)_{ij} \neq 0$ iff $\cos\theta_{i-1} \neq 0$ and $\cos\theta_j \neq 0$, which by definition of $S(\theta)$ is exactly the overlay condition $i \in R(S(\theta))$, $j \in C(S(\theta))$. □

The bridge can be read off concretely from the matrix support pattern. Returning to the running example $S = \{2, 4\}$ of Example 3.7, Figure 3 highlights the two kinds of nonzero entries promised by the theorem: the spine entries $(j + 1, j)$ on the subdiagonal (open circles), and the overlay entries at positions (i, j) with $i \in R$, $j \in C$, $i \leq j$ (solid dots at the row-column intersections).

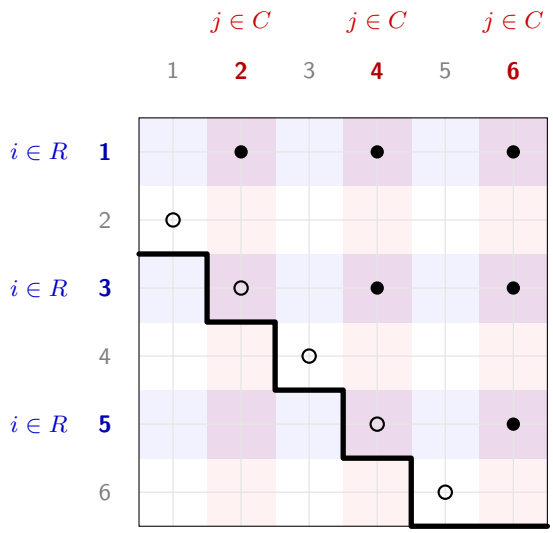

Figure 3: Support pattern of $Q_6(\{2,4\})$, the matrix-side counterpart of Figure 2. The thick boundary marks the upper Hessenberg band; open circles (○) are the mandatory subdiagonal spine entries; solid dots (●) are overlay nonzeros at the intersections (purple) of active rows (blue) and active columns (red). The Bridge theorem says these are exactly the nonzero entries of any unreduced $Q_n(\theta)$ with $S(\theta) = \{2,4\}$.

Example 3.10 ($n = 5$, $S = \{2\}$)**.** [✓ example_D5_S2]

The angle vector $\theta = (\pi/2, \pi/4, \pi/2, \pi/2)$ gives:

$$Q = \begin{pmatrix} 0 & \frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}} \\ -1 & 0 & 0 & 0 & 0 \\ 0 & -\frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}} \\ 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 \end{pmatrix}, \qquad A = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

The Hamilton cycle is $1 \to 5 \to 4 \to 3 \to 2 \to 1$. Since $R = \{1,3\}$ and $C = \{2,5\}$ are disjoint, the digraph is loopless (Figure 4); A is exactly the support pattern predicted by Theorem 3.9.

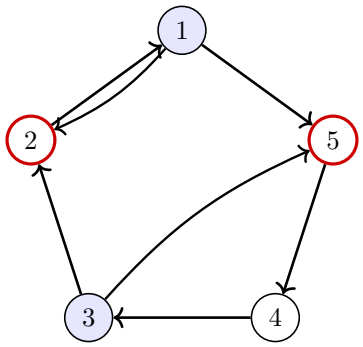

Figure 4: The loopless support digraph $D_5(\{2\})$. Active rows $R = \{1,3\}$ (blue fill) and active columns $C = \{2,5\}$ (red outline) are disjoint.

Theorem 3.11 (Rigidity)**.** [✓ rigid]

Let $S, S' \subseteq [n-1]$ with $|S|, |S'| \geq 2$. If $D_n(S) \cong D_n(S')$ as digraphs, then $S = S'$.

Proof sketch. The key combinatorial fact is the *cut lemma*: in $D_n(S)$, the only arc from $\{k{+}1, \ldots, n\}$ to $\{1, \ldots, k\}$ is the spine arc $k{+}1 \to k$. This forces every isomorphism to fix

the unique directed Hamilton cycle $1 \to n \to n{-}1 \to \cdots \to 2 \to 1$. A pigeonhole argument on the cut, combined with degree analysis at the maximum-in-degree vertex (which is n when $|S| \geq 2$), forces the isomorphism to fix n, after which a downward induction along the cycle forces the identity. The case $|S| = 1$ is the only failure: the cyclic rotation ρ^t exhibits $D_n(\{t\}) \cong D_n(\{n-t\})$, with no further coincidences. □

Example 3.12 ($n = 5$, $S = \{1, 3\}$). [✓ example_D5_S13]

A two-element example illustrating Theorem 3.11: any $D_5(S')$ isomorphic to $D_5(\{1, 3\})$ must have $S' = \{1, 3\}$. The angle vector $\theta = (\pi/4, \pi/2, \pi/4, \pi/2)$ gives:

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ -\frac{1}{\sqrt{2}} & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ 0 & 0 & 0 & -1 & 0 \end{pmatrix}, \qquad A = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Here $R \cap C = \{1\}$, so there is exactly one self-loop at vertex 1 (Figure 5).

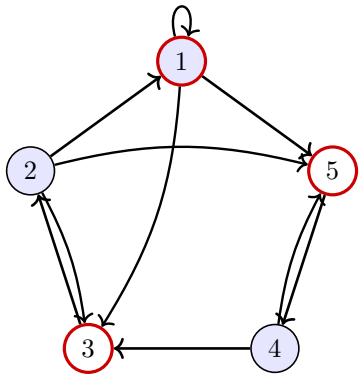

Figure 5: The support digraph $D_5(\{1, 3\})$. Since $R \cap C = \{1\}$, there is exactly one self-loop at vertex 1.

Theorem 3.13 (Classification). [✓ classification]

Fix $n \geq 2$. The classification of support digraphs of unreduced orthogonal upper Hessenberg matrices is the conjunction:

(1) (realizability) *every $S \subseteq [n-1]$ is of the form $S(\theta)$ for some $\theta \in \Theta_n$;*

(2) (bridge) *for every θ, the support of $Q_n(\theta)$ equals $D_n(S(\theta))$;*

(3) (rigidity) *for every S, S' with $|S|, |S'| \geq 2$, $D_n(S) \cong D_n(S') \Rightarrow S = S'$.*

The classification declaration is a one-line term-level packaging of three independently-proved components; it adds no mathematical content and exists only so the paper has a single Lean handle to cite. Realizability is witnessed by the explicit angle choice $\theta_k = \pi/4$ if $k+1 \in S$, $\theta_k = \pi/2$ otherwise; this gives $\sin\theta_k \neq 0$ always and $\cos\theta_k \neq 0$ exactly on S.

Theorem 3.14 (Connected count). [✓ card_isoClasses_quot]

For $n \geq 2$, the number of non-isomorphic weakly connected support digraphs of $n \times n$ unreduced orthogonal upper Hessenberg matrices is

$$N_n \;=\; 2^{\,n-1} \;-\; \left\lfloor \tfrac{n-1}{2} \right\rfloor.$$

Theorem 3.15 (Loopless count). [✓ card_looplessClasses_quot]

For $n \geq 3$, the number of non-isomorphic loopless connected support digraphs is

$$N_n^{\text{loopless}} \;=\; F_{n-1} \;-\; \left\lfloor \tfrac{n-3}{2} \right\rfloor,$$

where F_k is the k-th Fibonacci number ($F_0 = 0$, $F_1 = 1$).

Proof sketch (both counts). Both counts follow the same template: *labeled* configurations are subsets $S \subseteq [n-1]$ (resp. $S \subseteq \{2, \ldots, n-2\}$ with no consecutive elements, the loopless characterisation). The number of labeled configurations is 2^{n-1} (resp. F_{n-1} via the standard Fibonacci-counts-binary-strings bijection). Subsets with $|S| \geq 2$ are rigid (Theorem 3.11), so each is its own isomorphism class. The empty subset is rigid alone. The $|S| = 1$ subsets pair up under $\{t, n-t\}$, contributing $\lceil (n-1)/2 \rceil$ (resp. $\lceil (n-3)/2 \rceil$) classes. □

References

[ABB+23] Aaron Anderson, Mantas Bakšys, Jonas Bayer, Mauricio Collares, Rémy Degenne, Yaël Dillies, Ben Eltschig, Sébastien Gouëzel, Kalle Kytölä, Rob Lewis, Paul Lez, Luccioli Lorenzo, Heather Macbeth, Patrick Massot, Arend Mellendijk, Kyle Miller, Pietro Monticone, Kim Morrison, Oliver Nash, Utensil Song, Terence Tao, Floris van Doorn, Sky Wilshaw, and Lawrence Wu. Formalization of the Polynomial Freiman-Ruzsa Conjecture of Marton, November 2023.

[ACMT25] Jeremy Avigad, Johan Commelin, Heather Macbeth, and Adam Topaz. Anatomy of a formal proof. *Notices of the American Mathematical Society*, 72:1, 02 2025.

[Avi17] Jeremy Avigad. Modularity in mathematics, September 2017.

[Avi23] Jeremy Avigad. Mathematics and the formal turn. *Bulletin of the American Mathematical Society*, 2023.

[Avi26] Jeremy Avigad. Mathematicians in the age of ai, 2026.

[Baa24] Anne Baanen. Use and abuse of instance parameters in the lean mathematical library. *Journal of Automated Reasoning*, 69(1):1, 2024.

[BBB+25] Matthew Bolan, Joachim Breitner, Jose Brox, Nicholas Carlini, Mario Carneiro, Floris van Doorn, Martin Dvorak, Andrés Goens, Aaron Hill, Harald Husum, Hernán Ibarra Mejia, Zoltan A. Kocsis, Bruno Le Floch, Amir Livne Bar-on, Lorenzo Luccioli, Douglas McNeil, Alex Meiburg, Pietro Monticone, Pace P. Nielsen, Emmanuel Osalotioman Osazuwa, Giovanni Paolini, Marco Petracci, Bernhard Reinke, David Renshaw, Marcus Rossel, Cody Roux, Jérémy Scanvic, Shreyas Srinivas, Anand Rao Tadipatri, Terence Tao, Vlad Tsyrklevich, Fernando Vaquerizo-Villar, Daniel Weber, and Fan Zheng. The equational theories project: Advancing collaborative mathematical research at scale, 2025.

[BBC+26] Anne Baanen, Matthew Robert Ballard, Johan Commelin, Bryan Gin-ge Chen, Michael Rothgang, and Damiano Testa. Growing mathlib: maintenance of a large scale mathematical library. In Valeria de Paiva and Peter Koepke, editors, *Intelligent Computer Mathematics*, pages 51–70, Cham, 2026. Springer Nature Switzerland.

[BdFFD+25] Lars Becker, María Inés de Frutos-Fernández, Leo Diedering, Floris van Doorn, Sébastien Gouëzel, Asgar Jamneshan, Evgenia Karunus, Edward van de Meent,

Pietro Monticone, Jasper Mulder-Sohn, Jim Portegies, Joris Roos, Michael Rothgang, Rajula Srivastava, James Sundstrom, Jeremy Tan, and Christoph Thiele. A blueprint for the formalization of Carleson's theorem on convergence of Fourier series, 2025.

[BT25] Kevin Buzzard and Richard Taylor. FLT. https://github.com/ImperialCollegeLondon/FLT, 2025. An ongoing Lean formalization of Fermat's Last Theorem.

[CT24] Johan Commelin and Adam Topaz. Abstraction boundaries and spec driven development in pure mathematics. *Bulletin of the American Mathematical Society*, 61, 02 2024.

[Ili26] Vasily Ilin. Semi-autonomous formalization of the vlasov-maxwell-landau equilibrium, 2026.

[Li26] Xinze Li. Astrolabe: A content-addressable hypergraph for semantic knowledge management, 2026.

[Liq22] Liquid Tensor Experiment contributors. Liquid tensor experiment. https://github.com/leanprover-community/lean-liquid, 2022. Completed July 2022.

[LPSS26] Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The network structure of mathlib, 2026.

[mC20] The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, POPL '20, page 367–381. ACM, January 2020.

[MU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing.

[Par72] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[Sev03] Simone Severini. On the digraph of a unitary matrix. *SIAM Journal on Matrix Analysis and Applications*, 25(1):295–300, 2003.

[vDEL20] Floris van Doorn, Gabriel Ebner, and Robert Y. Lewis. Maintaining a library of formal mathematics. In Christoph Benzmüller and Bruce Miller, editors, *Intelligent Computer Mathematics*, pages 251–267, Cham, 2020. Springer International Publishing.

[vDMN23] Floris van Doorn, Patrick Massot, and Oliver Nash. Formalising the h-principle and sphere eversion. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2023, page 121–134, New York, NY, USA, 2023. Association for Computing Machinery.